Архитектура объектового хранилища.

Денис Черемисов

12 марта 2026 г.

Аннотация

Данная статья содержит описание архитектуры объектового хранилища ориентированного на масштабирование, высокую локальную и глобальную производительность и качественную диагностируемость за счёт простой схемы взаимодействия компонентов, каждый из которых четко изолирован в рамках своего домена ответственности.

Содержание

1 Определения. 4

2 Физическое устройство системы. 5
2.1 Слой горячего хранения. . . . 5
2.1.1 Максимальная производительность. . . . 5
2.1.2 Высокая производительность. . . . 6
2.1.3 Вариант для дешёвых on-premise решений. . . . 6
2.2 Слой холодного хранения. . . . 6
2.3 Выбор кортежа. . . . 7
2.4 Мета 8
2.4.1 Records 8
2.4.2 Fragments 8
2.4.3 Links 8
2.4.4 Containers 9
2.4.5 Tuples 9

3 Процессы. 10
3.1 Вид издалека. . . . 10
3.1.1 Запись. . . . 10
3.1.2 Чтение. . . . 11
3.2 Создание индексов. . . . 11
3.3 Дедупликация. . . . 11
3.4 Процесс обычной закачки данных. . . . 12
3.5 Процесс потоковой закачки данных. . . . 12
3.6 Процесс чтения. . . . 12
3.7 Миграция из жары в холод. . . . 13
3.8 Уплотнение данных и GC. . . . 14
3.8.1 Уплотнение данных на страйпах. . . . 14
3.8.2 GC. . . . 14
3.8.3 Коллизии переупаковки и GC. . . . 16

4 Инварианты системы 18
4.1 Инварианты ссылочной целостности 18
4.2 Инварианты миграции Hot → Cold 18
4.3 Инварианты WAL и повторов 19
4.4 Инварианты дедупликации и GC 19
4.5 Инварианты исполнения 19

5 Дополнительные материалы. 21
5.1 Полный WritePath для нескольких фрагментов. . . . 21
5.2 Полный MultiWrite. . . . 21
5.3 О реализации WAL с брокером и KV-хранилищем. . . . 22
5.3.1 Вводные. . . . 22
5.3.2 Работа с KV. . . . 22
5.3.3 Работа с брокером. . . . 23

6 Специализированное решение для WAL. 23
6.1 Консенсус-узел. . . . 23
6.2 Клиенты. . . . 24

7 Roadmap. 25
7.1 MVP. 25
7.2 GC. 25
7.3 Развитие логики. 25
7.4 Оптимизация. 25
7.5 Описание механизма «продолжения». 26

1. Определения.

Система делится на следующие компоненты:

- Координаторы без состояния.
- Холод, делящийся на холодные узлы:
  - RAFT аллокатор выделения кортежей и *страйпов* на них.
  - Кортежи дисков обслуживаемые аллокатором.
- Жара, она представлена узлами из трёх машин в идентичной конфигурации.Возможны несколько вариантов в зависимости от требований к цене и производительности.
- WAL, распределённый WAL, пока проще всего реализовать с помощью брокера и KV хранилища с персистетностью и гарантиями. Например, на основе Nats + JetStream, который и брокер и KV.
- Метаданные:
  - Записи Records с метаданными записей.
  - Фрагменты Fragments с метаданными фрагментов включащих поля поддержки дедупликации.
  - Коллекция Links для связи Records и Fragments.
  - Containers для описания физического размещения контейнеров с фрагментами: контейнер может быть «запечатан» в страйпе холодного слоя либо пока лежать на буфере горячего слоя.
  - Tuples для связи кортежей холодного слоя с дисками

2. Физическое устройство системы.

2.1. Слой горячего хранения.

Здесь я предлагаю три разных подхода, отличающихся экзотичностью железа и ценой. Во всех случаях в горячем слое мы используем максимальную степень избыточности. Т.к. на этом этапе проще всего работать от «наибольшего знаменателя», т.е. использовать схему ×3, а уже в холодном хранилище понижать степень избыточности. Т.е. когда мы говорим о «горячем узле» мы имеем в виду три узла, каждый из которых хранит копию данных. И соответствующие накопители на машинах должны быть эквивалентными с точки зрения аппаратного LBA и объёма.

В любом из режимов диск D_i представляется в следующем виде:

$$D_i = (B_{i1}, B_{i2}, \dots, B_{im})$$
$$|B_{i1}| = |B_{i2}| = \dots = |B_{im}| = 64\text{MiB}$$

Блоки B_{ij} называются *буферами.*

2.1.1. Максимальная производительность.

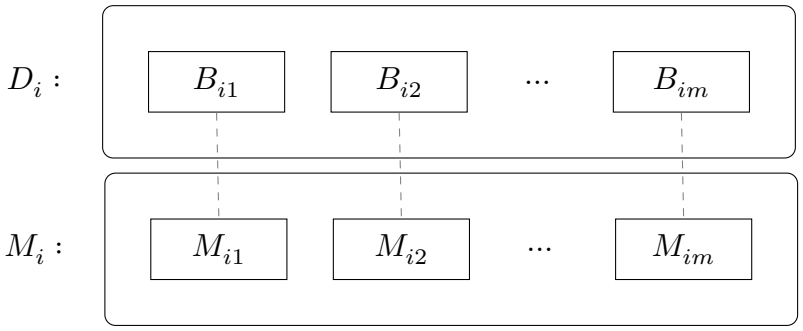
Аппаратные требования:

- Один или несколько NVMe SSD.
- Плашка/плашки NVDIMM (дешевле чем Optane при схожих гарантиях).

В этом режиме мы используем NVMe SSD. Один, два, три. Больше вряд ли имеет смысл, т.к. три накопителя сожрут любую сеть, а NVMe на PCI-e x5 сожрет в одиночку.

И для каждого диска D_i на NVDIMM плашках выделяется область памяти M_i, которая так же делится на равные фрагменты M_{ij}, где $j = 1, 2, \dots, m$:

$$M_i = (M_{i1}, M_{i2}, \dots, M_{im})$$
$$|M_{i1}| = |M_{i2}| = \dots = |M_{im}|$$



Размер фрагментов M_{ij} определяется исходя из физического размера блока на дисках. Каждый M_{ij} хранит:

- метаданные контейнера заполняемого (или заполненного) в буфере B_{ij}.
- буфер в смысле буферизованного IO для незаполненных при записи LBA.

При последующих записях этот буфер будет заполнен до конца и улетит в хвост B_{ij}.

Обратим вниманием на выбор буферов для записи. Мы должны смещаться от начала диска до его конца при выборе. Т.е. все блоки B_{ij} будут заполняться почти последовательно. Это обеспечит равномерный износ накопителя. И при неудавшейся записи мы ни в коем случае не должны пытаться вновь писать в тот же блок. Потому что это будет попытка перезаписи и это будет напрасным износом диска. Просто проскакиваем область LBA которая должна была содержать фрагмент и всё. Т.е. фундаментально придерживаемся append-семантики.

Такой подход позволяет обслуживать сотни видео-стримов в высоком разрешении.

2.1.2. Высокая производительность.

Аналогично предыдущему пункту, но без NVDIMM. В этом случае метаданные буферов будут храниться в какой-либо базке, а вот упаковки в LBA не будет. Т.е. если запись не поместилась в блок накопителя целиком, то остаток блока заполняется нулями и следующая запись начинается с начала следующего блока.

В целом, такой подход будет обеспечивать достаточно высокую производительность, хотя уступающую прыдущему варианту. Но для видео-стримов подходить будет хуже, придётся заморачиваться с буферизацией по LBA на координаторах. Что явно нехорошо, т.к. это будет требовать знания кишок жары.

2.1.3. Вариант для дешёвых on-premise решений.

Диск как кольцевой буфер.

Здесь мы вообще не заморачиваемся более-менее плотной упаковкой буферов и если записываемый фрагмент выходит за границы буфера, то мы сразу запечатываем буфер и готовим его к перевозу в холод, а фрагмент пишем уже на следующий. Такой подход противопоказан стримам, т.е. такая жара подходит только для всяких хранилищ документов. Но зато она дешёвая и может быть реализована на любом железе, даже на HDD.

2.2. Слой холодного хранения.

Он разбивается на холодные узлы и каждый узел это пара:

- RAFT-аллокатор.
- Набор кортежей дисков.

Кортеж T представляет собой набор дисков

$$T = (D_1, D_2, \dots, D_N)$$

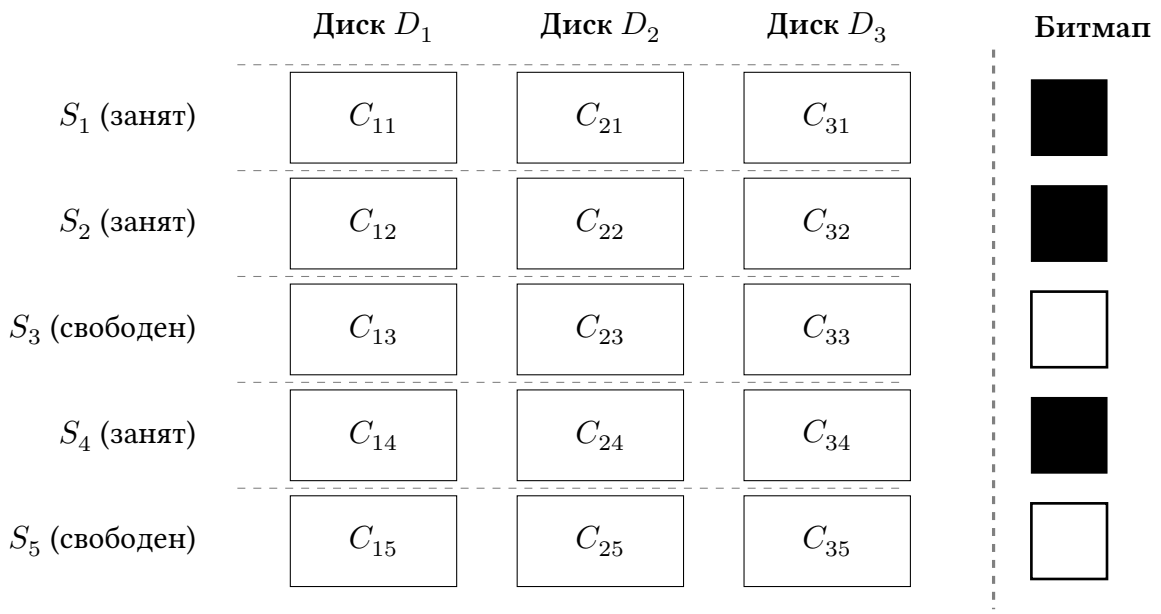
Сами диски представляются как наборы блоков одинакового между собою размера:

$$D_i = (C_{i1}, C_{i2}, \dots, C_{iM})$$

Страйпом S_j в кортеже T называется набор блоков с индексом j набранных со всех дисков кортежа:

$$S_j = \left(C_{1j}, C_{2j}, \dots, C_{Nj}\right)$$

Задачей RAFT-аллокатора является поиск свободных страйпов в обслуживаемых кортежах. Т.е. вначале он выбирает один из кортежей и затем ищет свободный страйп. Учёт свободный-несвободный ведётся с помощью *битмапа* кортежа, где каждому страйпу сопоставлен бит, имеющий значение 1 для занятых и 0 для свободных страйпов:



Выделение страйпа производится в два шага:

1. Lease страйпа.
2. Подтверждение занятия страйпа (Commit) или отмена занятия (Release).

Release производится автоматически по TTL, если от координатора ведущего миграцию не было никакого сигнала в течении Δ секунд. Таким образом, для перемещения 64МиБ данных у нас проводятся 2 RAFT-операции. Это 64 RAFT-операции в секунду на проводку потока данных в обе объёме 2ГиБ в секунду. Это ничтожно мало, тем более в условиях когда задержки нас не особо волнуют т.к. это фоновый процесс.

2.3. Выбор кортежа.

При выборе кортежа важно склоняться к выбору наименее заполненных и не слишком сильно загруженных в данный момент. Следующий алгоритм должен быть рабочим.

Есть некоторый параметр – натуральное число N. Кортежи хранятся в структуре

```
tuples = RBTree[FreeStripes, LinkedList[Tuple]]
```

Т.е. кортежи сгрупированны по количеству доступных для записи страйпов в них и упорядочены по количеству этих страйпов. Выбор страйпа под запись осуществляется следующим образом:

1. Выбираем случайный K из $[0, N)$.
2. Берём K-ый кортеж из структуры и запускем там процесс записи данных.
3. После подтверждения записи, изымаем кортеж из связного списка с данным весом и переносим в конец списка с весом ниже.

Можно пропускать сильно нагруженные в данный момент кортежи, вообще не учитывая их, в поиске K-го. Это приведёт к увеличению сложности поиска, но мы говорим о всего 32 таких операциях на 2Гб/с.

2.4. Мета

В качестве БД для хранения метаданных лучше всего подходит TiKV: практически неограниченное горизонтальное масштабирование, высокая производительность, особенно в rawkv режиме.

2.4.1. Records

record.id	single	fragment.id	...

record.id	multi	no_of_fragments	...

Хранит метаданные записей. Здесь мы в случае одного фрагмента в записи храним ссылку на него прямо в records, чтобы избежать избыточного хранения данных. В случае хранилища мелкодокументов получим полуторакратное уменьшение числа записей в метаслое: не будет записей в Links.

2.4.2. Fragments

fragment.id	container.id	offset	size	crc64	compression type	blake3	...

Хранит метаданные фрагментов.

2.4.3. Links

record.id	fragment.index	fragment.id

Хранит связи между записями и фрагментами:

rid	0	fid_0
rid	1	fid_1
rid	2	fid_2
	...	
rid	n	fid_n

На самом деле лучше, вместо 0, 1, ...имеет смысл хранить SPECK32(i), для предотвращения ситуаций, когда один шард получает большее число записей с большим числом фрагментов. За счёт такого трюка записи разъедутся по шардам. Мы, конечно, потеряем возможность скана по коллекции для вычитки последовательности фрагментов при этом, но нужда в нём не является насущной: все фрагменты кроме последнего имеют размер около 64Мб и скорость чтения из БД метаданных здесь не будет являться узким местом.

А код итерации с

```
for x in records_fragments_iterator:
  read_fragment(x.fragment_id)
```

Изменится на

```
for x in [0, record.no_of_records):
  fragment_id := records_fragments.fragment(record.id, speck32(i))
  read_fragment(fragment_id)
```

2.4.4. Containers

container.id	ref

Где ref может быть Hot(hot.id, block.index) или Tuple(tuple.id, stripe.index).

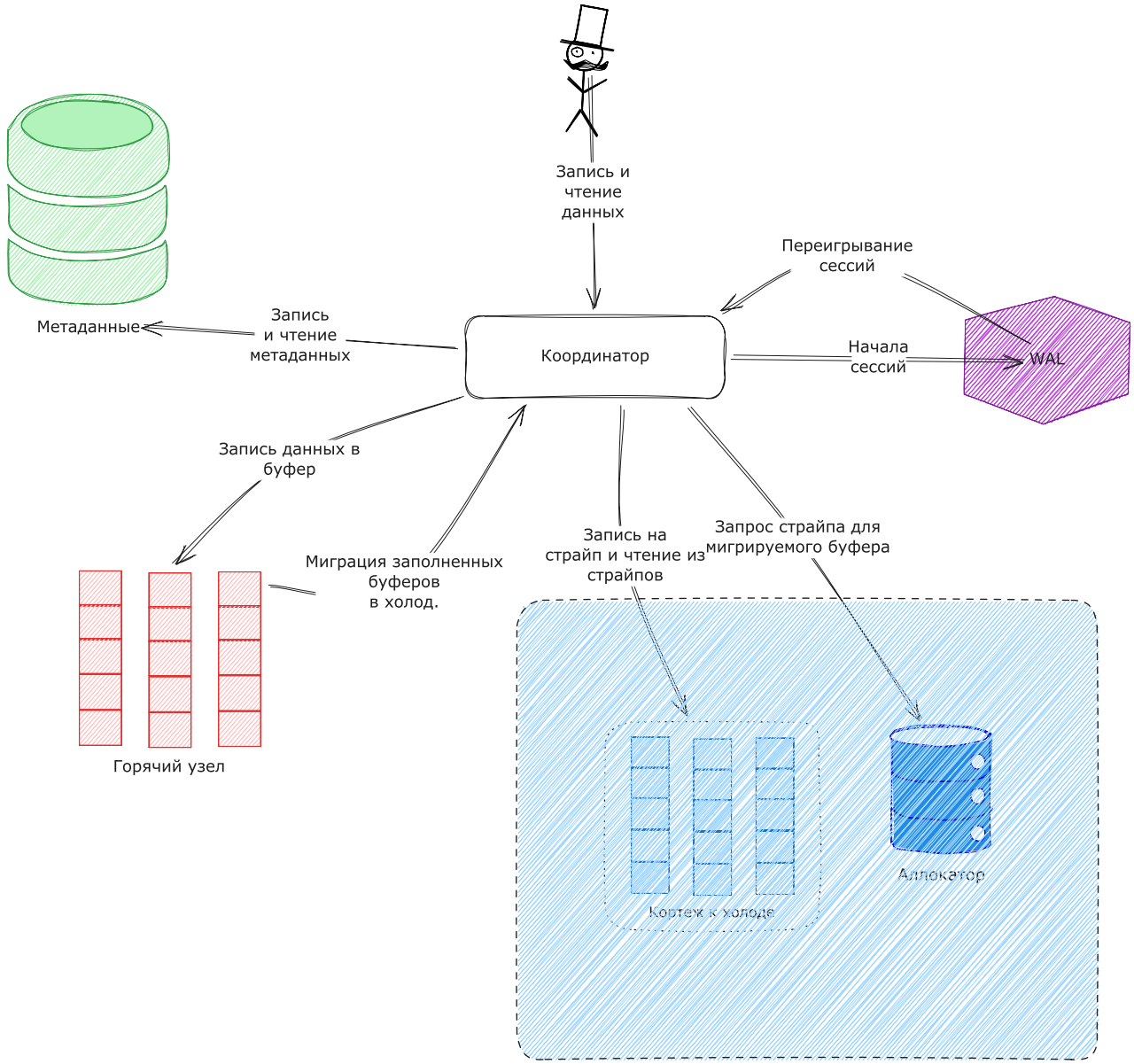
2.4.5. Tuples

tuple.id	payload

С *payload* вроде такого

```
{
    type: тип используемого кодирования
    disks: [
       (disk1, role1),
       ...
       (diskN, roleN),
    ]
}
```

3. Процессы.



3.1. Вид издалека.

Если не углубляться в подробности, схема работы выглядит следующим образом:

3.1.1. Запись.

1. Клиент льёт данные к координатору.
2. Координатор дробит поток на фрагменты и пишет их жару, а метаданные в БД метаданных.
3. Координатор возвращает клиенту идентификатор записи.

Запись в жару может приводить к периодическим сигналам от неё, приводящим к миграции данных из жары в холод:

1. Жара сообщает мигратору (может быть и координатором) что буфер готов к переносу.

2. Мигратор вытаскивает данные из жары, находит свободный страйп в холоде и пишет туда полученные данные, а затем отображает это в метабазе.

3.1.2. Чтение.

1. Клент спрашивает у координатора данные записи.
2. Координатор по БД метаданных определяет их расположение и читает из холода и/или жары.

3.2. Создание индексов.

Заводится etcd хранящий u128 для record.id, fragment.id, container.id и т.д. При старте координатор спрашивает у этого etcd диапазон для record.id, скажем 2^{20} чисел. После этого он выделяет их по-одному. Далее, чтобы обеспечить равномерное разделение по шардам, мы используем не прямое значение идентификаторов, а кодированое значение SPECK128(id), которое выдаёт хорошее равномерное распределение значений. В качестве альтернативы мы можем добавлять hash(id) в таблицы, но это будет стоить некоторых объёмов. Лучше использовать SPECK128.

3.3. Дедупликация.

Данные фрагмента содержат поля

CRC64	Size	CompressionType	Blake3

Можно держать коллекцию следующего вида

CRC64	Size	CompressionType	Blake3	fragment.id	epoch

И при записи фрагмента искать уже существующие фрагменты по ней. Если есть, то смотреть на эпоху:

- Если она слишком далеко в прошлом, то не используем, а пишем новый фрагмент и перезаписываем эту запись новой ссылкой.
- Иначе данные не пишем, а используем существующий фрагмент.

Такая процедура на самом деле может приводить к некоторому дублированию данных. Это другой процесс проделывает то же самое одновременно: пишет фрагмент и заменяет дедуплицирующую запись. Таким образом, мы получим два фрагменты с идентичным содержимым. Но это больше синтетическая проблема. В реальных сценариях такое должно быть крайне нетипичным, а сценарий приводящий к массовому распуханию вообще представляется синтетическим.

3.4. Процесс обычной закачки данных.

При обычной загрузке данных, когда пользователь загружает известный набор данных, вся запись R делится на фрагменты:

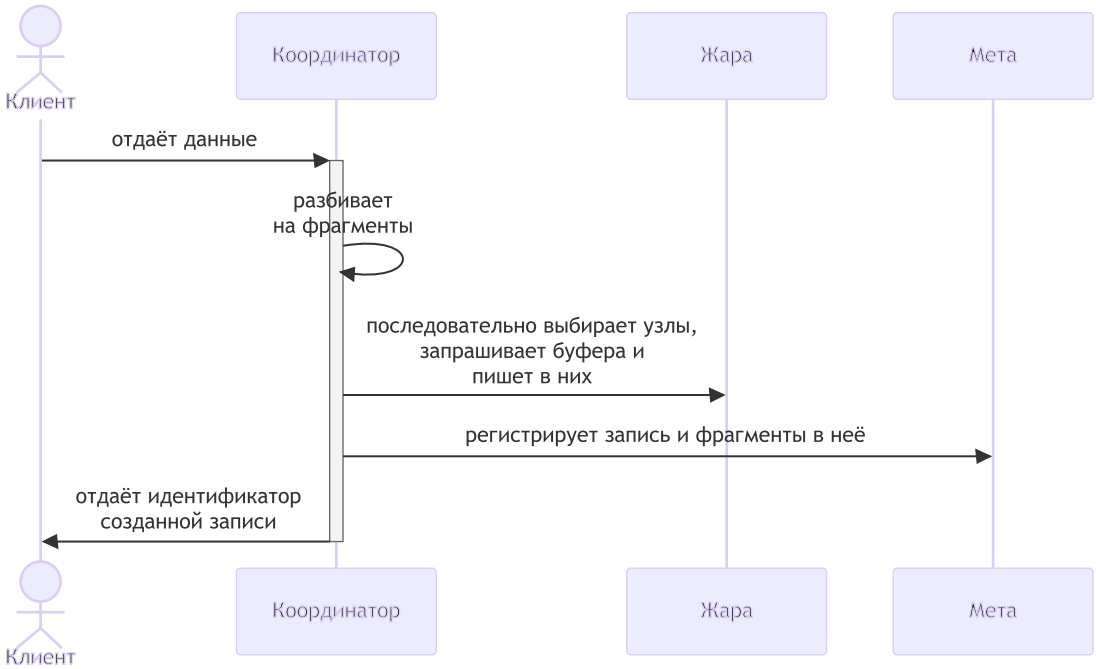
$$R = (F_1, F_2, \dots, F_n)$$

где длины фрагментов строго совпадают вплоть до последнего, который может быть меньше остальных. Т.е., формально,

$$|F_1| = |F_2| = \dots = |F_{n-1}| \geqslant |F_n|$$

Такое ограничение введено для простоты поддержки чтения с отступом. Можно, конечно, было бы разбивать фрагменты на блоки произвольного размера, но это значительно бы усложнило стоимость реализации.

При отдаче пользователем данных – упрощённо, на самом деле там возня с WAL и последовательность обращений к БД метаданных – виде происходит следующее:



В конце материала можно увидеть полную формальную последовательность WritePath с поддержкой фиксации или отката.

3.5. Процесс потоковой закачки данных.

В это режиме мы берём буфер в жаре во временное владение с подтверждением. И льём туда данные до заполнения. После заполнения запечатываем буфер и ставим задачу на перевоз в холод. Из явных отличий от обычной закачки данных нужно отметить ведение фрагмента: содержимое фрагмента в Fragments помечается как «in progress», и часть метаданных этого фрагмента (размер и т.п.) хранится в метаданных буфера. Это нужно чтобы избежать слишком интенсивных записей в БД метаданных в процессе.

3.6. Процесс чтения.

Поддерживается полное чтение и чтение с отступом. По пунктам:

1. Получаем record по выданному RecordID.

2. Читаем идентификаторы фрагментов сканируя Links по ключу (RecordID, <индекс фрагмента>)

3. Для каждого фрагмента читаем его метаданные из Fragments и находим ContainerID.

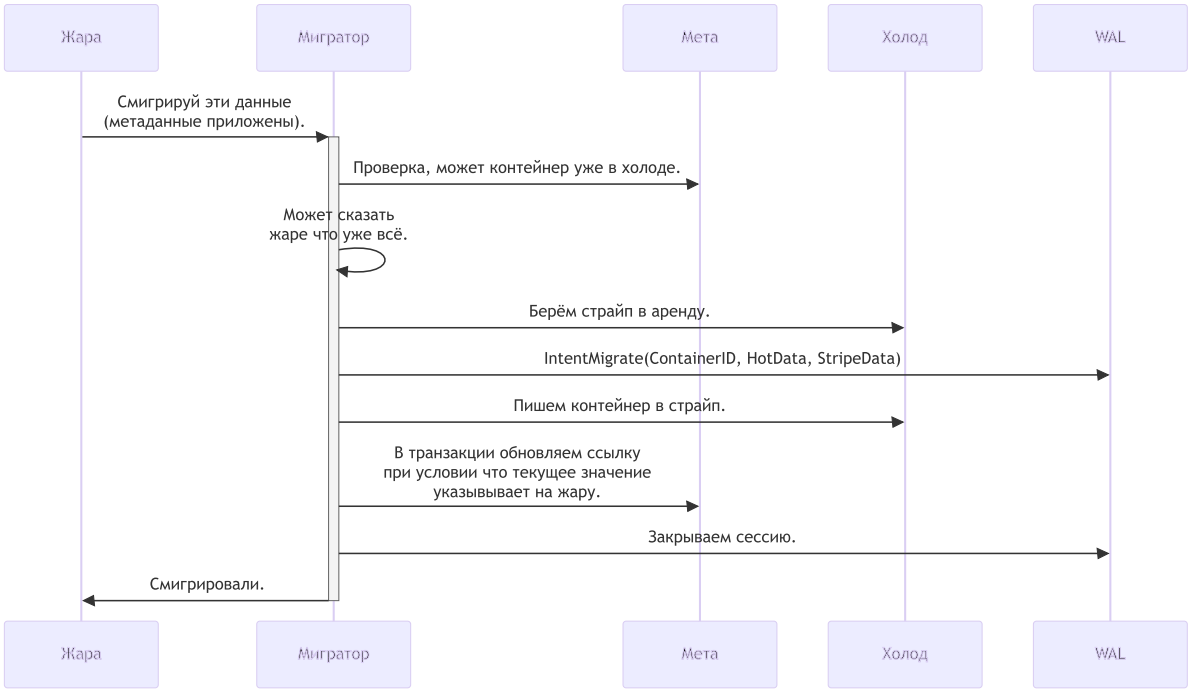
4. Читаем метаданные контейнера из Containers по ContainerID и находим по нему физическое размещение, в холоде ли или в жаре.

5. Если контейнер в жаре, то делаем запрос на чтение из буфера жары.

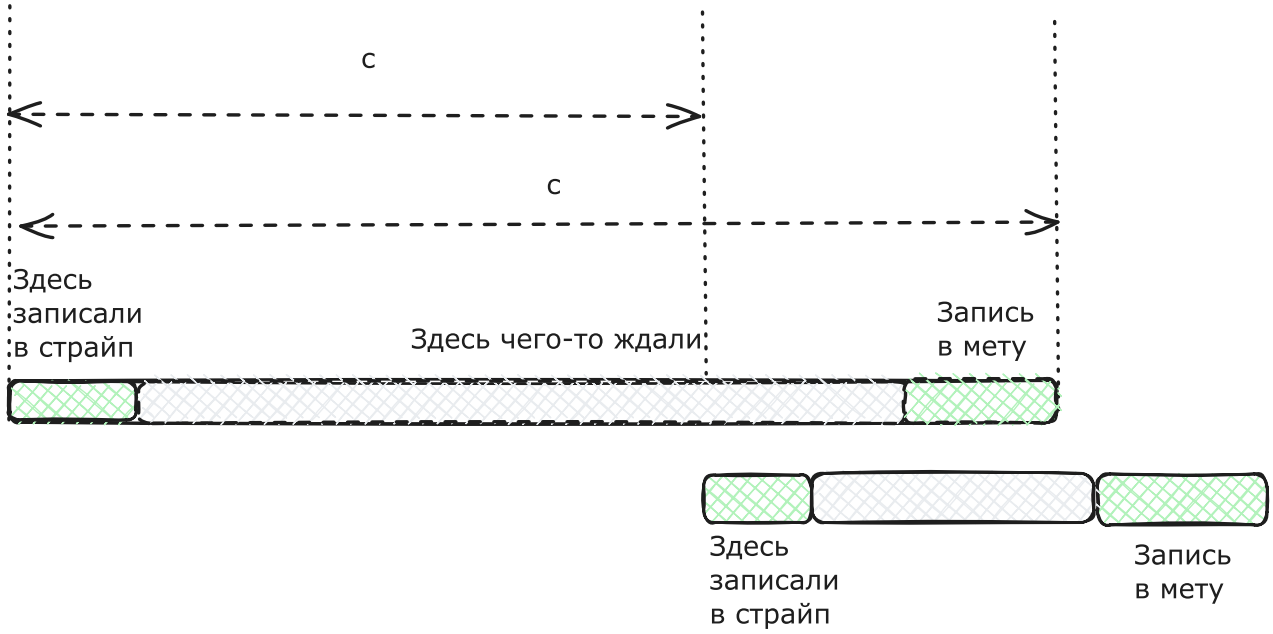
6. Если контейнер в холоде, т.е. адресуется по (TupleID, StripeIndex), то запрашиваем данные кортежа из Tuples и вычисляем по смещению фрагмента в контейнере нужные блоки.

3.7. Миграция из жары в холод.

Когда буфер не имеет смысла больше заполнять, т.к. на нём осталось слишком мало свободного места, жара начинает процедуру миграции, обращаясь для этого к **мигратору.** Происходит это по следующей схеме.



Использование транзакции необходимо чтобы избежать появляение сирот, как указано в этой схеме:



Когда предыдущий процесс может очень сильно затормозить и бысть сочтенным жарой мёртвым. Но остаться в живых и приступить к записи уже после повторной попытки миграции. Транзакции здесь не будут слишком обременительными: одна транзакция соответствует записи 64Мб данных. Это смешные для TiKV 32 транзакции при 2Гб/с записи в систему.

Данные сохранённые в WAL позволят освободить занятый в текущей сессии страйп, если буфер был перенесён другой миграцией.

3.8. Уплотнение данных и GC.

3.8.1. Уплотнение данных на страйпах.

Уплотнение страйпов заключается в персборке N страйпов в M страйпов, где $M < N$. В метаданных контейнера должны хранится следующие данные: объём незанятого места (не удалось записать фрагмент во время наполнения или фрагмент был удалён – место фрагмента считается свободным) и средний размер фрагмента на диске.

Наивный алгоритм компактизации выглядит следующим образом:

Бежим по записям компактизуемых страйпов и читаем фрагменты, сохраняя в буфере в памяти. В ходе этого процесса, если следующая не лезет в буфер целиком (размер буфера = размеру блока в жаре), то выделяем под него страйп и сохраняем там.

Не наивный чуть посложнее, с переупорядочиванием фрагментов и ведением нескольких буферов за раз. Для того чтобы достичь наилучшего заполнения каждого.

В общем, несложная алгоритмическая задача. Естественно, в процессе регистрируем намерения в WAL, чтобы при потенциальном повторе понимать, с какими страйпами мы имеем дело. После успешной перупаковки или сразу после сохранения буфера в страйпе проводим процедуру изменения ссылок в Fragments на новые.

3.8.2. GC.

Это самая сложная тема здесь, пожалуй. Как понятно из всего материала сверху, подчистка ведётся на уровне избавления от фрагментов, которые никем более не используются. И она актуальна только для системы с дедупликацией. В случае отключенного дедупа GC не будет представлять никаких сложностей и реализуется путём фонового скана кортежей в холодном хранилище, кортеж за кортежом. Фрагменты готовятся к удалению когда на них

больше не осталось ссылок в Links. Процедура определения этого факта весьма тяжела если делать напрямую:

- Хранение обратного индекса fragment.id -> (record.id, индекс фрагмента). Работать такое, скорее всего будет, но базке от такого точно будет нелегко. Потому что здесь O(1) от Links, а это самая большая коллекция у нас. Но, в принципе, в инсталляциях относительно небольшого размера такое возможно.

- Полное сканирование Links. Это тяжело и ненадежно: никто не гарантирует доступность всех шардов во время скана базы. Опять же, в небольших инсталляциях это возможно.

- Подсчёт ссылок и хранение времени доступа. Достаточно надёжно: refcount $\leqslant 0$ и фрагмент не трогался уже длительное время – с большой вероятностью он больше ни на что не указывает. Но такой подход требует перезаписей метаданных фрагментов (можно делать раз в сутки: видим что чисталось менее 24 часов назад – не трогаем время последнего чтения).

Здесь у каждого из вариантов есть существенные недостатки. Можно утверждать, что первые два точно непригодны для «exascale». А вот как раз для огромных массивов данных можно применить типичный для этого «exascale» подход с офлайн-обработкой. Для этого нужно кидать события создать запись и «привязать» фрагмент к записи в сторонее хранилище. Например, в Clickhouse, или в хранилище использууемое Spark/Flink (на самом деле кидать будем в Kafka или RedPanda, а уже оттуда оно будет подсасываться в Clickhouse).

Скажем, если это Clickhouse, то создание записи можно отражать с помощью (datetime, record.id, 1). Удаление – (datetime, record.id, 0). И сделать record.id – первичным ключом в таблице. И если использовать какой-нибудь CollapsingMergeTree, то первая запись будет заменена на вторую после добавления последней.

Затем мы можем делать LEFT OUTER JOIN с таблицей отражающей Links по record.id при условии флаг = 1. Тогда record.id будет NULL для записей в Links, если запись была удалена. И тогда мы можем пометить такие фрагменты как предназначенные для удаления. Точнее, предлагается подход с «даунгрейдом» фрагмента:

1. Созданный фрагмент имеет состояние $LIVE$.

2. Фрагмент в состоянии $LIVE$ попавший под удаление переходит в состояние $DYING$. Оно может быть выражено появлением поля $death_time$.

3. Фрагмент в состоянии $DYING$ попавший под удаление на сей раз удаляется, если текущее время больше чем $death_time$.

Правда, для этого нужно запретить "старые" фрагменты для дедупа. Скажем, если подходящий по хешу фрагмент старше 3-х месяцев, то создавать новый фрагмент, даже если это будет дублем.

Такая схема будет полностью рабочей, при условии что рамках записей данные будут лететь как в базу метаданных, так и в "окно" для Clickhouse. Скажем, мы записали запись и связь записи с фрагментом в метабазу. После этого сразу выходим, но сессия продолжает работать в рамках сессии мы добавляем эти же данные в CH. В случае неудачи записи сессия отправляется на повтор и мы снова пробуем добавить эти данные.

3.8.3. Коллизии переупаковки и GC.

В описанных выше идеях не были упомянуты возможные коллизии. Так, например, две процедуры уплотнения могут взять один и тот же контейнер и переместить его «живое» содержимое, естественно по разным контейнерам. Что в итоге создаст сироту:

1. Уплотнение номер 1 изменяет для фрагмента fid текущий страйп A на созданный в процессе B.

2. Уплотнение номер 2 делает то же самое, изменяя ссылку из A в C.

В итоге система теряет любое знание о том, что B содержит фрагмент с данным fid, место занимаемое им помечено как использование. Мы получили потерю пространства.

А в случае наложения работы переупаковки и GC возможна коллизия, когда проводится процедура компакции и одновременно довозятся фрагменты на удаление. Фрагмент fid переводится в другой страйп и в то же самое время доставка из GC удаляет его, причём последовательность выстраивается следующим образом:

1. GC-процесс узнает какой контейнер содержит данный фрагмент.

2. GC-процесс уменьшает описание свободного места в контейнере.

3. Переупаковка изменяет ссылку в фрагменте.

4. GC-процесс удаляет фрагмент из Fragments.

В итоге, контейнер содержащий переупакованные данные содержит неправильную информацию о свободном месте.

В обоих этих случаях можно было бы работать через транзакции, но это может быть слишком дорого. Но всех вариантов коллизий можно избежать, если гарантировать, что никакие две процедуры уплотнения не пересекаются. И что никакая текущая процедура уплотнения не пересекается с отражением результатов GC. Я предлагаю следующий подход:

- Заводится очередь задач. Она содержит задачи на уплотнение и на удаление. Проще всего сделать это через брокера.

- Заводится сервис накидывающий задачи уплотнения в очередь.

- Заводится сервис получающий данные от GC и подготавливающий задачи на удаление.

Исполнитель задачи получивший её вначале пытается взять $lease$ на неё. $lease$ - это запись в TiKV вида $high \to (low, high)$. Для этого он в выполняет в транзакции.

1. Поиск $lease'$ с минимальным $high' \geqslant high$. Если $low' \leqslant low$, то это коллизия. Задача отправляется на переповтор в будущем, а $lease$ не создаётся.

2. Иначе создаём $lease$ и приступаем к выполнению задачи.

3. Никаких TTL! Потому что мы можем остаться в ситуации, когда данные перевезены, но метаданные только частично. При разрешении начать работу над незавершенными данными могут появиться сироты – занятые контейнеры. На части которых никто не указывает и которые не помечены незанятыми. Операция будет проводиться под WAL-ом и этого достаточно, система докатит изменения и в конце снимет аренду.

Как ставятся задачи уплотнения: сервис итерируется по регионам, нарезая каждый на куски случайной длины, но находящейся в определенном диапазоне. Если в регионе последний кусок вышел меньше минимального, то захватывается поддиапазон следующего региона. При этом, что важно, рассматриваются только поддиапазоны следующего региона с индексами больше чем самый большой в диапазоне текущего – это потому что регионы «под капотом» могут изменяться в процессе. Множество регионов рассматривается как кольцевой буфер, так что «следующий» для послдеднего будет первым регионом в TiKV.

Сервис-адаптер для GC тем временем читает выхлоп Clickhouse-а, получая идентификаторы фрагментов для «даунгрейда». Затем находит регионы в которых лежат контейнеры и группирует фрагменты по регионам. Деля, при необходимости, регионы на внутренние диапазоны одинакового размера – так проще всего. Балансируя между количеством задач и размером диапазона для $lease$: чем больше задач, тем меньше должна быть длина. Для того чтобы сократить время $lease$ и чтобы сама задача весила поменьше. Проще (и правильнее) всего будет на стороне Clickhouse-а упорядочивать выхлоп фрагментов по возврастанию индекса.

4. Инварианты системы

Данный раздел фиксирует свойства, которые должны оставаться истинными при любых сценариях работы системы, включая сбои, повторы WAL-сессий и миграции. Сюда намеренно не включены состояния, существующие только в рамках конкретных алгоритмов выполнения (например, $Hot \to Migrating \to Cold \to Compacted$ для контейнера). Эти состояния не являются частью модели данных системы и могут изменяться вместе с реализацией. Инварианты фиксируют только проверяемые свойства персистентного состояния, достаточные для обеспечения корректности хранения и доступности данных.

4.1. Инварианты ссылочной целостности

I1. Единственная истина о размещении. Физическое размещение фрагмента определяется исключительно через цепочку:

$$Fragments[fragment.id].container.id \to Containers[container.id].ref$$

Никакие другие коллекции не содержат авторитетной информации о физическом размещении.

I2. Контейнер имеет ровно одно размещение. Для любого $container.id$ значение ref принадлежит одному из двух состояний:

$$ref \in \{Hot(hot.id, block.index),\ Tuple(tuple.id, stripe.index)\}$$

Промежуточные или двойные состояния в метаданных запрещены. ref ссылается на кортеж и страйп существующий в коллекции $Tuples$ и битмапе аллокатора.

I4. Полнота записи single определяется идентификатором фрагмента в Records. Состав записи $record.id$ в формате single определяется идентификатором фрагмента в $record.single(fragment$

I4. Полнота записи multi определяется только Links. Состав записи $record.id$ определяется исключительно строками:

$$(record.id, fragment.index) \to fragment.id$$

Коллекция $Records$ не содержит авторитетной информации о составе записи в формате multi.

I5. Фрагмент является неизменяемым. После фиксации записи поля:

$$(offset,\ size,\ crc64,\ compression,\ blake3)$$

не могут изменяться. Любое изменение содержимого приводит к созданию нового $fragment.id$.

I6. Целостность кортежа Для любого $tuple.id$ в $Tuples$ количество дисков в массиве $disks$ и их роли должны соответствовать схеме, заявленной в $type$.

4.2. Инварианты миграции Hot → Cold

M1. Завершённость миграции определяется сменой ref. Миграция контейнера считается завершённой только после атомарной замены:

$$Hot(\cdot) \to Tuple(\cdot)$$

в $Containers.ref$.

M2. Commit страйпа после смены ref. Страйп считается занятым (Commit) только после:

1. успешной записи всех блоков,
2. успешной подмены $Containers.ref$.

M3. Эксклюзивность записи страйпа. Запись в $(tuple.id, stripe.index)$ допускается только при активном Lease, выданном RAFT-аллокатором. Lease на страйп (tuple.id, stripe.index) выдаётся аллокатором только в том случае, если конфигурация дисков в кортеже из Tuples позволяет выполнить запись (например, все диски живы и доступны).

M4. Идемпотентность миграции. Если $Containers.ref$ уже имеет вид $Tuple(\cdot)$, повтор миграции обязан завершаться без побочных эффектов.

M5. Возможность смены ref. Атомарный переход возможен только как $ref_expected \to Tuple(\cdot)$, где $ref_expected$ — точное текущее значение $Containers.ref$ (Hot или композитный Hot, т.е. не $Cold$).

4.3. Инварианты WAL и повторов

W1. Все многошаговые операции выполняются через WAL. Любая операция, затрагивающая более одного домена (жара, холод, мета), должна выполняться внутри WAL-сессии.

W2. Повтор не порождает дубликатов. Повтор WAL-сессии не должен создавать дополнительных ссылок, страйпов или контейнеров. Повторная операция либо:

- перезаписывает те же значения,
- либо обнаруживает завершённое состояние и завершает выполнение.

4.4. Инварианты дедупликации и GC

D1. Dedup влияет только на выбор fragment.id. Дедупликация не изменяет модель ссылок. Связи записи с фрагментами всегда выражаются через $Links$.

G1. Удаление фрагмента допустимо только при отсутствии ссылок. Фрагмент может быть удалён тогда и только тогда, когда не существует строк в $Links$, указывающих на данный $fragment.id$.

4.5. Инварианты исполнения

E1. Запись в блоки страйпа и в буфер возможна только при живом Lease. Любая операция записи в cold-страйп или в буфер жары допустима только при наличии действительного $lease.id$, выданного соответствующим координатором (RAFT-аллокатором или узлом жары). При утрате или истечении Lease операции записи становятся невозможными.

4.X. Инварианты владения и конкурентного исполнения

O1. Единственный владелец диапазона. В каждый момент времени существует не более одного процесса, обладающего правом модификации контейнеров диапазона R. Право определяется действительным $lease.id$, выданным менеджером диапазонов.

O2. Модификация контейнера только через очередь диапазона. Любая операция, изменяющая состояние контейнера (включая перераспределение фрагментов, изменение метрик свободного места, результаты компакции или GC), выполняется исключительно как задача очереди соответствующего диапазона его владельцем.

Q1. Линейность исполнения задач диапазона. Задачи диапазона исполняются последовательно одним исполнителем. Каждая задача имеет уникальный идентификатор $(rangeID, taskID)$. Повторная постановка или выполнение задачи не нарушает корректность (идемпотентность или обнаружение повторного выполнения).

Q2. Монотонность прогресса. Для каждого диапазона существует монотонная точка прогресса ack. Значение ack может изменяться только вперёд. Восстановление после перезапуска выполняется путём продолжения с последнего зафиксированного ack.

G2. Актуальность инъекции GC. При применении задачи InjectGC$(container, F)$ для каждого $f \in F$ действие допустимо только если на момент исполнения выполняется

$$\text{Fragments}[f].container_id = container.$$

В противном случае операция является *no-op*.

G3. Отсутствие прямой записи GC в контейнеры. Процесс GC не выполняет прямых модификаций структуры контейнеров. Он формирует только задачи диапазонов, исполняемые их владельцами.

M6. Фиксация размещения перед коммитом ресурса. Любой коммит ресурса cold-слоя допустим только после успешного атомарного перехода

$$ref_{expected} \to Tuple(\cdot),$$

выполненного в транзакции. При неуспешном переходе ресурс не фиксируется.

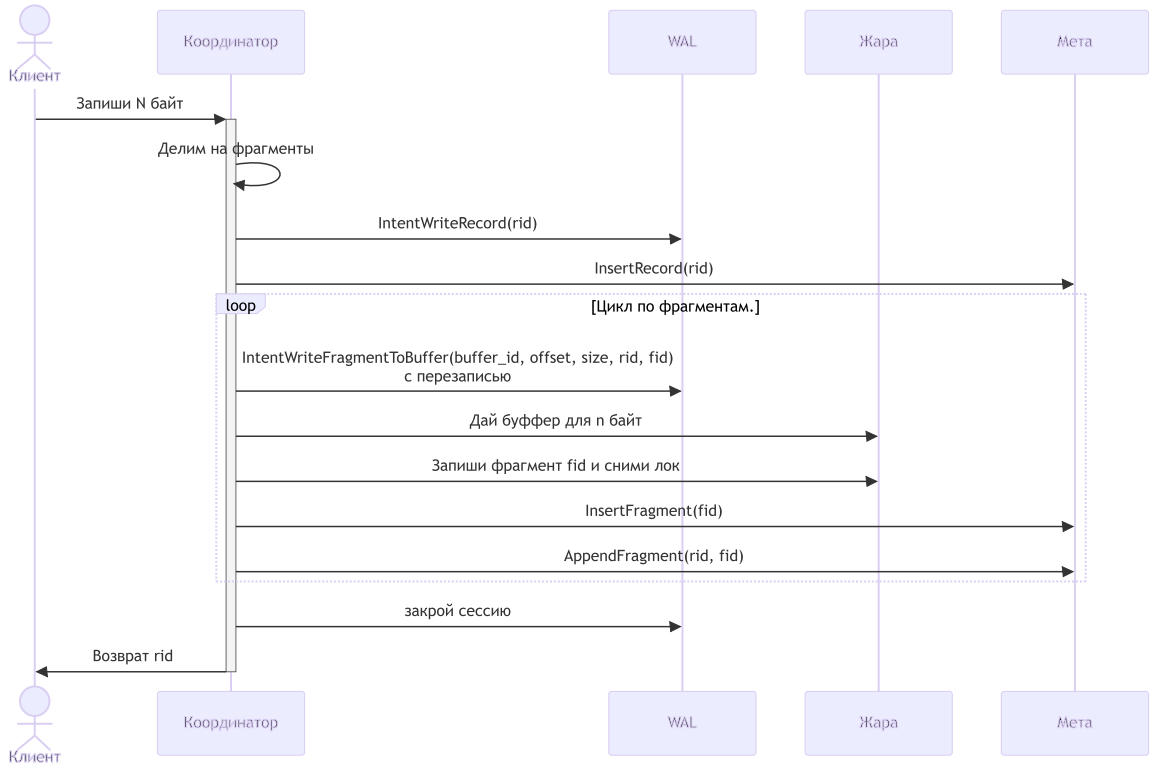
ID1. Каноничность ключевого идентификатора. Во всех структурах, где требуется равномерное распределение, используется ключ

$$id_{key} = \text{SPECK}_{128}(id_{raw}),$$

где SPECK_{128} — биективная перестановка. Это обеспечивает статистически равномерное распределение контейнеров по диапазонам.

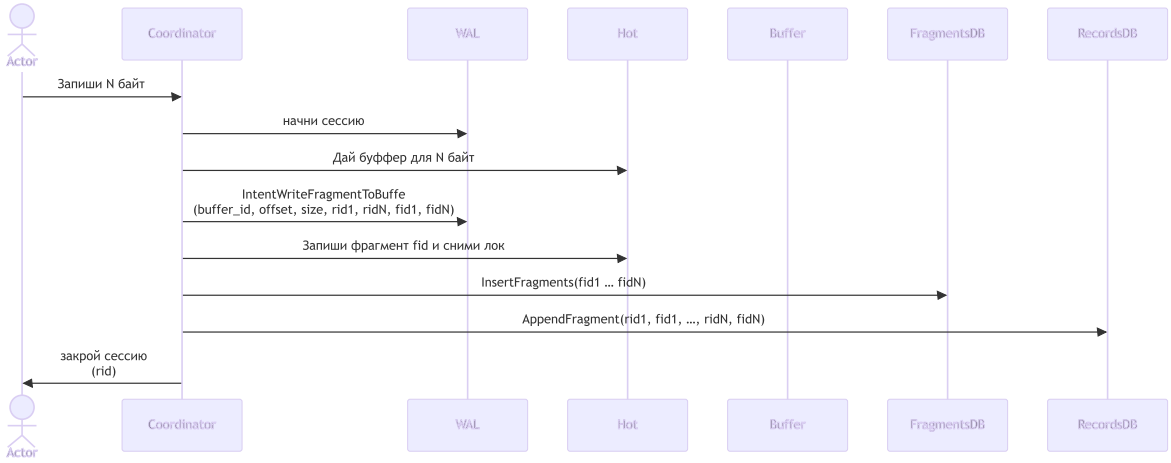
5. Дополнительные материалы.

5.1. Полный WritePath для нескольких фрагментов.



Из хорошего здесь то, что операции с БД не должны быть транзакционными, инварианты обеспечиваются WAL-ом и идемпотентностью логики.

5.2. Полный MultiWrite.



Вставка нескольких фрагментов сразу. Это место критичное для производительности системы при вставке мелких данных. При ней повышенную нагрузку получает мета-слой, в который будет лететь куча записей. Схема описанная здесь снимает нагрузку на WAL – использование диапазонов (rid1, ridN) и (fid1, fidN) для сохранения намерений в WAL. И нагрузку на хранилище метаданных за счёт использования пакетных записей.

Обратите внимание, что в качестве rid1 ...ridN и fid1 ...fidN использованы "оригиналы" идентификаторов, а не их SPECK128 значения. Как раз для того, чтобы мочь использовать диапазонную нотацию, а не перечисления. В случае ошибок в сессии разбор будет вестись уже с преобразованными значениями, естественно. Т.е. проверяя ridK из rid1 ...ridN мы на

самом деле проверяем запись с идентификатором SPECK128(ridK).

5.3. О реализации WAL с брокером и KV-хранилищем.

Реализуется распределенный WAL держащий по логу на сессию. В рамках работы с сессией доступны следующие операции:

- Добавить данные – Append.
- Перезаписать данные - Overwrite.
- Закрыть сессию.
- Отправить сессию на повтор с некоторой задержкой от T секунд.

Предлагается реализовать эти операции с помощью брокера и KV-хранилища. В качестве брокера предлагается использовать NATS + JetStream. В качестве KV всё что угодно. Из персистентных лучше всего подойдут In-Memory хранилища вроде тарантула или Redis. Из хранящих на диске хороший кандидат - Aerospike.

Вариант хранилищ с LSM использующих RocksDB под капотом не подойдёт категорически. Т.к. наша нагрузка подразумевает

- Постоянное создание новых записей, без повтора идентификаторов.
- Удаление абсолютного большинства записей в доли секунды.
- Большая часть записей переживших те доли секунды всё равно будет удаляться через секунды.

В этих условиях skip list в RocksDB хранящий таблицу LSM в памяти является крайне неоптимальным вариантом из-за своего ленивого характера удалений. Он будет приводить к сильнейшей нагрузке на IO в дальнейшем.

5.3.1. Вводные.

При заводе сессии координатор получает свежий session_id, далее просто id, о 16 байтах (128 бит). В этих условиях вводим:

$$K = \text{big_endian}(id)$$
$$K' = \text{append}(K, 0)$$

Т.е. к K' просто прибавили нулевой байт.

5.3.2. Работа с KV.

При начале работы сразу же запускается фоновый процесс регулярно пишущий unixtime в K', с выставленным TTL = T. С периодичностью, скажем, в одну секунду. Чаще не нужно. А данные сессии пишутся следующим образом:

header	data

Где хидер содержит в т.ч. число повторов совершенных с этой сессией, а данные - последовательность бинарных данных в формате

varint(len1)	data1	varint(len2)	data2	...	varint(lenN)	dataN

Все бинарные данные сессии всегда содержатся у координатора. В этих условиях

- Операции Append соответствует добавление к данным сессии нового varint(len(data)) + data и последующая запись всего вместе с хидером в значение K
- Операции Overwrite соответствует удаление всех данных после хидера и добавление нового фрагмента, с последующей записью.

Когда сессия завершается с успехом, значение K удаляется из БД. Если сессия была неуспешной, просто выходим. Неудача обновления K' считается неудачей работы с сессией.

5.3.3. Работа с брокером.

На определённый топик брокера подписаны все координаторы. Распостранение событий не broadcast, а доставка только одному из подписчиков.

В брокер, в этот же топик, летит событие содержащее только K, с указанием доставить не ранее чем через T.

При получении события могут развиваться по следующим сценариям.

- В базе нет записи с ключом K. Это означает, что сессия была завершена. Ack.
- Значение в K' было слишком недавно. Заказываем передоставку через T.
- Счётчик повторов в записи K слишком велик. Сообщаем про это в логе, удаляем K и делаем Ack.
- Предыдущие пункты не выполнились. Начинаем восстановление сессии запуская обновлялку K', увеличиваем счётчик повторов на один (сразу сохраняя новое значение) и начинаем работу по данным записанным в сессии. Далее всё как было описано в пункте «Работа с KV.»

6. Специализированное решение для WAL.

Предлагается вариант с Push-Push взаимодействием работающим следующим образом:

6.1. Консенсус-узел.

Консенсус-узел в WAL состоит из $2n+1$ машины с налаженными между ними MultiRAFT или аналогичным консенсус-протоколом, с $2n+1$ внутренним RAFT-ом, где каждый хост является лидером одного из них и ведомым для остальных RAFT-ов.

Каждый хост реализует интерфейс вида

```
rpc Session(stream SessionReq) returns (stream SessionResp)

type SessionReq oneof {
  Init(clientClass) // Только первый в потоке.
  Append(bytes)
  Overwrite(bytes)
  Close             // Только последний в потоке.
```

```
  Repeat(T seconds) // Только последний в потоке.
  Pulse             // Произвольный в потоке.
}
```

SessionResp содержит подтверждения для проведенных операций, каждый ответ ожидается синхронно.

Клиент подключается к **конкретному** хосту такого узла и инициализирует его с помощью Init, затем выполняет последовательность Append/Ovewrite и завершает финальным Close/Repeat. При этом в фоне периодически ходит Pulse, так же с подтверждением.

Активные сессии хранятся в памяти. Сессии оправленные на повтор сохраняются в LSM упорядоченном по времени повтора

repeat_time	headers	data

Если отвалился клиент, то сессия отправляется на повтор. Если отвалился хост, то бывший ведомый ставшим лидером сессии, немедленно отправляет активные сессии полученного RAFT-а на повтор. Повтор сессии начинается по консенсусу.

6.2. Клиенты.

Сервисы, обеспечивающие обработку повторов сессий, должны реализовывать почти зеркально-симметричный интерфейс

```
rpc SessionRepeat(stream SessionRepeatReq) returns (stream SessionRepeatResp)
```

Где SessionRepeatResp выглядит почти в точности как SessionReq, а вот SessionRepeatReq получает новую ветвь, в сравнении с SessionResp:

```
type SessionRepeatReq oneof {
  Deliver(session_headers, session_data) // Только первый в потоке.
  // Далее в точности как SessionResp.
}
```

При повторе хост узла определяет класс сервисов на которых нужно вести повтор сессии и подключается для исполнения, начиная с доставки данных сессии. Это и есть второй Push в «Push-Push» схеме.

7. Roadmap.

7.1. MVP.

1. Реализация RAFT-аллокатора.
2. Реализация сервиса представляющего доступ к дискам машины висящим на JBOD на Go и io_uring.
3. Tuples и доведение до ума логики привязки дисков к кортежу (машина может меняться).
4. Тестирование заливки блоков в "холодный узел".
5. Реализация выбранного типа хоста горячего хранилища на Go + io_uring.
6. Реализация узла горячего хранилища.
7. Обвязка для WAL.
8. Сбор всей логики для меты и выбор базы.
9. Реализация координатора.

7.2. GC.

Присмотреть реализацию GC для малых инсталляций и для «exascale» с Clickhouse через Kafka в сторонке.

7.3. Развитие логики.

1. Группировка кортежей в аллокотаре по типам используемой избыточности. Когда аллокатор рассматривает только кортежи сделанные под определенную EC или репликационную схему.

7.4. Оптимизация.

1. Реализация хоста горячего хранилища с NVMe и NVDIMM на SPDK, выбрать C или Rust.
2. Реализация узлов с JBOD-ами на Rust с io_uring.
3. Замена WAL с костыльного NATS + JetStream на специализированный Push-Push WAL с мультимастером.
4. Внедрение архивного слоя, где данные и их метаданные сжимаются в рамках одного контейнера, а TiKV хранит только bloom-фильтры и координаты кортежей и страйпов архивного слоя. Разложение данных по страйпам идентично холодному слою, но оперирует меньшими (по P) схемами EC и большими размерами страйпов, вроде 1ГиБ вместо 64МиБ. Это позволит снять нагрузку с TiKV, который может начать задыхаться на большом количестве мелкозаписей, при котором вес метаданных будет приближаться к весу самих данных.

7.5. Описание механизма «продолжения».

При стримах у нас может отваливаться буфер. В этом случае можно искать другой буфер в другом месте, и писать в него не сначала, а с позиции обрыва записи на поломанный. При переводе склеивать первый и второй буферы по позиции обрыва. Отражая это, естественно, в метаданных.